Seeing through the canopy: recovering apple-tree leader structure from summer RGB-D images

MMA3001 Numerical Methods and Machine Learning, Individual Project. Shreeji (Jude) Kumawat, 37573578. Monash University, Semester 2 2026.

Repository: github.com/shreejikumawat/MMA3001 (main branch). Every number in this report is produced by a script in that repository; the README lists the command for each one.

Summary. A compact U-Net that reads one summer RGB-D image at 64 × 64 px recovers the trunk and two leaders of 2D open-V apple trees on orchard rows it never saw in training. It scores a centreline F1 of 0.295 ± 0.007 (3 seeds) against 0.120 for a classical ridge-and-depth baseline, measures the leader opening angle to a median error of 4.7°, and runs in 6 ms on one CPU thread of a laptop. A data audit found that the supplied masks are displaced from the images by about 10 px on most trees, which caps every strict score, so a shift-compensated score (0.462) is reported beside it. Smaller inputs generalised better than larger ones (64 > 128 > 256 px), depth helped only at 64 px, and fork height missed its 10 px target.

1. Engineering problem

A pruning or thinning robot in a trellised apple orchard has to know where the woody structure of each tree is before it moves an arm into the canopy. In summer much of that structure is behind leaves: in this dataset 37% of the labelled centreline is covered by foliage (Section 2).

Context. The orchard uses a 2D open-V trellis, where each tree is a short trunk that forks into two leaders trained into a flat V. The planar shape is what makes robotic pruning and harvesting practical, since an arm can reach the whole canopy from the inter-row. To plan an approach the robot needs two numbers per tree, the fork height and the opening angle of the V, plus a rough 3-D centreline to avoid. Depth cameras (an Intel RealSense D435 in the source study) are cheap, but a depth return from a leaf says nothing about the branch behind it.

Problem and why it matters. Given one summer RGB-D image of one tree, estimate the trunk and leader structure including the hidden parts, and report fork height, leader angle and a 3-D skeleton. Hand pruning and thinning are among the largest labour costs in apple production and labour is scarce; mapping structure in summer, not only in leafless winter, lets one perception system support summer pruning and thinning.

Intended use. A perception module run once per tree on the robot's on-board computer before path planning. It must be accurate enough that a planned approach does not hit a leader and fast enough to keep pace along the row (target well under 1 s per tree on a CPU).

Scope and limits. Summer images only (winter images are used in the audit, not as inputs). Main structure only (trunk and two leaders), because that is what the ground truth labels; laterals and spurs are out of scope. Relative 3-D only, because the depth is an 8-bit rescaled image of unknown scale. A 2D open-V tree form is assumed; the structural measurements are not defined for other forms.

2. Inputs, outputs and data

Item	Format	Units and domain	Meaning and use	Invalid or missing input
Summer RGB (in)	PNG, 640 × 480 × 3, uint8	0 to 255 per channel	Colour image of one tree from the inter-row	Wrong shape or unreadable file raises ValueError
Summer depth (in)	PNG, 640 × 480, uint8 (3 identical channels)	Relative; larger = farther; 0 = no return	Separates the tree from the next row	Zeros kept as missing and passed as a separate validity channel
Branch probability (out)	float32, H × W	0 to 1, over the 480 × 480 centre crop	Per-pixel belief that trunk or leader is present, visible or not	n/a
Branch mask (out)	bool, H × W	Probability above a threshold chosen on validation rows	Region the arm must avoid	n/a
Fork row (out)	int	Pixel row, 0 = top	Height of the trunk split	None if no qualifying junction
Leader angle (out)	float	Degrees, 0 to 180	Opening of the V	None if a leader has < 10 skeleton pixels
3-D skeleton (out)	(K, 3) float	X, Y relative; Z in raw 8-bit depth units	Centreline for path planning	Pixels with no valid depth in a 7 × 7 window dropped

Function-level inputs, outputs and exceptions are documented in Google-style docstrings and the pdoc HTML documentation (doc/html).

Dataset. The TreeID dataset (Chen et al., 2020) was supplied by the teaching team. Each tree has five files: summer RGB, summer depth, a 256 × 256 summer ground-truth (GT) mask, winter RGB and winter depth. Following the source study, images are centre-cropped to 480 × 480 and resized. No data is committed to the repository (data/README.md explains how to obtain it), and the MIT licence covers the code only.

Data audit (scripts/run_audit.py). Before any modelling I checked the data. The findings, and what each changed, were:

Check	Finding	Consequence
Inventory	562 trees in 20 rows (6 to 44 per row), 2,810 files, all 5 modalities present	All 562 usable
Naming	42 row-4 winter files carry a second tree id before the time stamp	First parser skipped them silently; fixed and covered by a test
Duplicates (MD5)	6 groups of byte-identical winter depth maps, all in row 4	Winter depth not used as an input
Depth direction	Canopy reads larger than ground at the bottom of the frame in 99.6% of trees, so larger = farther	Depth rank-normalised with near = 1; the initial "near = bright" assumption was wrong
Missing depth	16% of summer pixels have no return (5th to 95th percentile: 7% to 26%)	Validity channel added
GT form	One connected trunk plus two leaders; 7.5% of pixels; branch width 11.0 px (SD 1.0) at 256 px	Metrics use a 4 px tolerance (Section 4)
Occlusion	37% of GT centreline under foliage (excess-green > 0.05), 16% to 60% across trees	Occluded-branch recall reported separately
Registration	GT displaced horizontally from the visible trunk; best-fit offsets have median magnitude 10 px, both signs	Strict scores are capped; a shift-compensated score is reported beside them

The registration finding has a likely cause. Chen et al. state that their masks were drawn on images taken after harvest, while the summer images supplied here still carry fruit, so each mask was probably drawn on a different capture of the tree and is displaced by a tree-specific amount. This is a hypothesis I have not been able to confirm with the dataset owners, but it is consistent with every observation above.

3. Computational approach

Two methods are built and compared: a four-parameter classical detector that encodes what a branch looks like, and a compact U-Net that learns it, including where hidden leaders run. Both feed the same structure-extraction step.

3.1 Preprocessing (preprocess.py)

Columns 80 to 559 (the centre 480 × 480) are resampled to 64, 128 or 256 px. Colour and depth use area averaging, which acts as an anti-aliasing filter when down-sampling; masks use nearest-neighbour so no new label values are invented (unit week 3, interpolation). Depth zeros are kept as missing and valid values are rank-normalised per image, which removes the unknown per-image 8-bit scaling. The excess-green index on chromaticity coordinates, ExG = 2g − r − b, separates leaves from bark independently of brightness.

3.2 Baseline: classical ridge and depth detector (baseline.py)

After zeroing pixels with ExG above 0.05, each pixel gets the score s = (w_r R + w_n N) / (w_r + w_n) and is labelled branch if s > τ. R is the Sato multi-scale ridge response (eigenvalues of the Hessian, a second-order finite-difference operator, at scales 1 to 3 px) normalised by its 99.5th percentile, and N is the depth-foreground rank. Components below a minimum size are removed. The parameters (w_r with w_n = 1 − w_r, τ, minimum size) are grid-searched over 75 combinations on half of the training trees, maximising centreline F1. It is fast, interpretable, needs no training data and represents how the problem was approached before deep learning. Its built-in limit is that it can only find visible bark, and measuring that gap is the point of the comparison.

3.3 Main method: compact U-Net (model.py, train.py)

- **Architecture.** U-Net (Ronneberger et al., 2015) with 3 pooling stages and 16 to 128 channels: 0.48 M parameters, 0.29 GFLOP per 64 px image. It is small because there are only a few hundred labelled trees and it must run on a robot CPU.
- **Inputs compared.** RGB (3 channels) against RGB-D (RGB + depth rank + validity, 5 channels), to measure what depth contributes.

- **Loss.** Binary cross-entropy with a positive-class weight (background/branch ratio from training data only) plus soft Dice. Plain BCE drifts towards predicting background because branches are only 7.5% of pixels.
- **Optimiser and stopping.** AdamW, learning rate 2×10^{-3}, weight decay 10^{-4}, batch 8, at most 40 epochs, early stopping after 8 epochs without improvement in validation centreline F1, best weights restored.
- **Augmentation and threshold.** Horizontal flips only (vertical flips would show trees growing downwards). The output threshold is chosen between 0.20 and 0.80 on validation rows. All seeds are fixed and main runs are repeated with 3 seeds.

3.4 Structure extraction (skeleton.py) and label registration (align.py)

The mask is reduced to its one-pixel skeleton. The fork is the lowest junction (3 or more skeleton neighbours) above which the skeleton splits into two pieces of at least 20 px each. The skeleton above the fork is split at the fork column into left and right leaders, each is fitted by linear least squares as column = a · row + b (unit week 5), and the angle between the two direction vectors is reported. Fitting column on row keeps the problem well conditioned because the leaders are near-vertical. For the 3-D skeleton each pixel is lifted with the pinhole model, $X = (u - c_x) Z / f_x$, with Z the median valid depth in a 7 × 7 window; without the depth scale and intrinsics for this crop, Z stays relative.

To study the label offsets, a mask is aligned to a reference image by maximising their cross-correlation. This is computed with the FFT in $O(N \log N)$ rather than $O(N^2)$ by direct summation, with zero-padding to prevent wrap-around and three-point parabolic interpolation of the peak for sub-pixel precision.

Key assumptions. (1) The 480 × 480 centre crop matches the mask field of view. (2) Larger 8-bit depth means farther, with 0 as missing (verified by the audit). (3) Trees in the same row are not independent, so splits are made by whole rows. (4) ExG > 0.05 marks foliage; this is used for occlusion statistics and the baseline, not by the U-Net.

4. Validation and verification design

What is validated. Branch masks against the hand-labelled GT (withheld test data); fork row and leader angle from the predicted mask against the same quantities from the GT mask; generalisation across the orchard through a grouped split; and the code itself through tests with analytical answers.

Grouped split by orchard row (splits.py). Trees in one row share lighting, background, camera pose and capture time, so a random image split would leak that and inflate scores (unit week 5.6). Whole rows are assigned to one partition, fixed in code before any training:

Partition	Rows	Trees	Used for
Training	the other 14 rows	398	Fitting weights; tuning baseline parameters
Validation	3, 13	67	Early stopping, threshold, and model selection
Test	5, 10, 15, 20	97	One final evaluation of each model

Validation information **did** influence model selection (stopping epoch, threshold, and the final choice of model, Section 6.1). Test information did not: test labels were never changed or used for tuning.

Metrics. All are computed at the GT's native 256 × 256, so models trained at different resolutions are compared on the same footing.

- **Centreline F1 at 4 px (primary).** Both masks are skeletonised; a true centreline pixel counts as recalled if a predicted centreline lies within 4 px, and vice versa for precision. Branches are 11 px wide, so 4 px accepts a centreline that stays inside the true branch. Pixel IoU is also reported, but it collapses for thin structures: a 2 px offset of a 1 px line gives IoU 0 and centreline F1 1 (a unit test demonstrates this).
- **Shift-compensated ("shape") F1.** The same score after the best horizontal re-registration of the label within ±24 px. It measures whether the predicted shape is right regardless of the label offset. It is optimistic by construction, so it is always shown beside the strict score.
- **Occluded-branch recall** (recall on GT centreline under foliage, following Chen et al.), **leader-angle error** (degrees) and **fork-row error** (pixels).
- **Uncertainty.** 95% bootstrap confidence intervals over test trees, spread over 3 training seeds, and Wilcoxon signed-rank tests on paired per-tree differences.

Verification of the implementation. The pytest suite (tests/, 53 tests, all passing; doc/test_report.txt) checks each numerical component against a case with a known answer:

Component	Manufactured or analytical check
IoU, Dice, tolerant F1	Hand-computed overlaps; a line shifted by k px is matched exactly when k ≤ tolerance

Component	Manufactured or analytical check
FFT cross-correlation	Equals direct summation; recovers imposed integer shifts exactly and a 5.4 px shift to within 0.2 px
Leader angle	Synthetic Vs with half-slopes 0.25, 0.5, 0.7 return 2·atan(a) to within 1.5°
Fork rule	Moves less than the old run-based rule when a synthetic mask is thickened
FLOP counter	Matches the hand count $2 \cdot H \cdot W \cdot C_{out} \cdot k^2 \cdot C_{in}$ for one convolution; scales 4× when the image side doubles
Training loop	Learns a trivial synthetic task (F1 > 0.9); two runs with the same seed give identical predictions
Back-projection and inputs	A constant-depth plane lands at that depth; wrong shapes, missing folders and unknown seasons raise clear errors

Acceptance criteria (set from the robot's needs before the final runs). (1) U-Net beats the baseline on centreline F1 with paired $p < 0.05$. (2) Median leader-angle error < 5° and median fork-row error < 10 px (about 4% of image height). (3) Inference < 1 s per tree on one CPU thread. (4) Seed-to-seed SD of centreline F1 < 0.03.

What this validation cannot establish. Absolute accuracy, because the labels are displaced from the images; metric 3-D accuracy, because there is no depth scale or 3-D ground truth; transfer to other orchards, cultivars, seasons or cameras, because all data come from one orchard and one camera; and behaviour on a moving robot (motion blur, changing light).

5. Results and comparison

U-Net values are means over 3 seeds ± seed SD; the baseline is deterministic. Brackets are 95% bootstrap intervals over test trees for seed 0. Inference is the network forward pass on one laptop thread (scripts/profile_pipeline.py); RGB times are approximate, as the network differs only in its first layer.

Method (test rows, 97 trees)	Centreline F1, strict	Shape F1	Occluded recall	IoU	Mean angle error	Inference
Classical baseline	0.120 [0.114, 0.125]	0.159	0.54	0.055	31.7°	65 ms
U-Net RGB, 128 px	0.252 ± 0.002	0.411 ± 0.021	0.79	0.217	11.8°	≈ 19 ms
U-Net RGB-D, 128 px	0.257 ± 0.009	0.431 ± 0.024	0.81	0.225	7.8°	19 ms
U-Net RGB, 64 px	0.245 ± 0.016	0.421 ± 0.014	0.84	0.230	8.0°	≈ 6 ms
U-Net RGB-D, 64 px (selected)	**0.295 ± 0.007** [0.261, 0.323]	**0.462 ± 0.006**	**0.87**	**0.260**	**6.3°** (median 4.7°)	**6 ms**

Paired Wilcoxon tests on per-tree strict F1 (seed 0 of each): the selected model beats the baseline by 0.170 ($p = 2 \times 10^{-14}$), the 128 px RGB-D model by 0.034 ($p = 0.036$) and the 64 px RGB model by 0.061 ($p = 4 \times 10^{-4}$).

- **Learning beats hand-crafted cues, mainly on hidden structure.** The baseline marks every dark ridge, including grass, shadows and trellis wire (centreline precision 0.07), and cannot see through leaves. The U-Net recovers 87% of occluded centreline against 54%.
- **Depth helps only at 64 px.** At 128 px RGB and RGB-D are indistinguishable (difference 0.006, $p = 0.65$); at 64 px depth adds 0.05 to 0.06. A plausible reason is that at 64 px each deep feature covers a larger part of the tree, so the coarse near/far layout that depth gives becomes useful context.

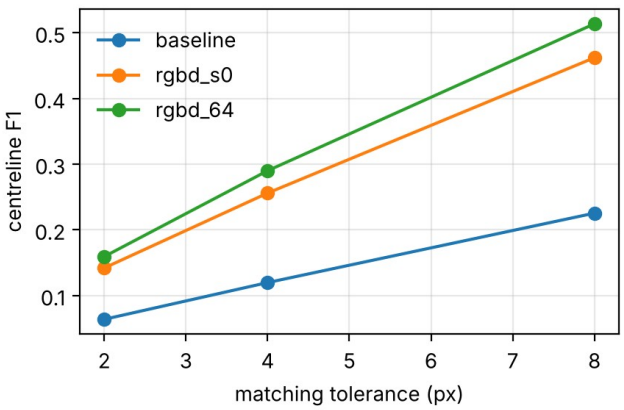


Figure 1. Score a perfect prediction receives when its label is shifted horizontally (97 test masks). At the 4 px tolerance, a 6 px offset drops the score from 1.0 to 0.06.

Why the strict scores look low. Figure 1 shows how fast a perfect detector loses score under a label offset. The shift-compensated scoring of the selected model finds a median offset of 7 px, with 68% of test trees beyond 4 px, so strict scores mostly measure label agreement and the shape score is the better measure of whether the predicted structure is right. Figure 2 (Appendix A) shows trees where the model follows the visible trunk and the hand label does not (R05N01, R15N08).

Acceptance criterion	Target	Result (selected model)	Met?
Beats baseline	$p < 0.05$	+0.170, $p = 2 \times 10^{-14}$	Yes
Leader angle	median < 5°	4.7°, 4.7°, 4.9° (3 seeds)	Yes, narrowly
Fork row	median < 10 px	17, 19, 11 px (3 seeds)	**No**
Inference	< 1 s per tree	about 16 ms end to end (6 ms network)	Yes
Repeatability	seed SD < 0.03	0.007	Yes

Comparison with the published benchmark. Chen et al. report a two-class mean IoU of 83.0% and occluded-branch recall of 78.5% for U-Net; the equivalent two-class mean IoU here is 0.54. The numbers are not comparable: their masks were scored against the images they were drawn on, whereas here they are displaced; their 50 validation images were also used to compare models, with no separate test set; and their occluded recall is a pixel recall on a hand-labelled occlusion class, not a centreline recall at 4 px. The gap is not evidence that this method is weaker, but it does show that the dataset as supplied cannot reproduce the published accuracy.

6. Alternatives, optimisation and performance

6.1 Optimisation path

Four changes were tested against the first working model. One of them, reducing the input to 64 px, improved every accuracy measure and halved network time, and was selected.

Version	Change and reason	Strict F1	Shape F1	Mean fork error	Decision
v1, 128 px	Class-weighted BCE + Dice (branches are 7.5% of pixels)	0.257 ± 0.009	0.431 ± 0.024	18.4 px	Starting point
v2, 128 px	Predicted masks were ~32 px wide against 11 px true; class weight removed	0.210 ± 0.043	0.360 ± 0.049	29.3 px	Rejected: masks only narrowed to ~26 px, one seed stopped after 1 epoch
v2r, 128 px	Training labels re-registered to a v1 prediction (6.3)	0.242 ± 0.002	0.450 ± 0.011	13.0 px	Not selected: better shape and fork, worse strict F1 and occluded recall (0.64 vs 0.81)
v1, 64 px	Input resolution reduced (6.2)	**0.295 ± 0.007**	**0.462 ± 0.006**	16.1 px	**Selected**

The 64 px configuration was first noticed in test numbers, which would be test-set reuse if adopted directly. I therefore re-ran the selection on the validation rows alone (scripts/validation_selection.py): 64 px RGB-D ranks first there too (strict F1 0.290 against 0.250 for the next family), so the choice does not depend on the test set.

6.2 Input resolution

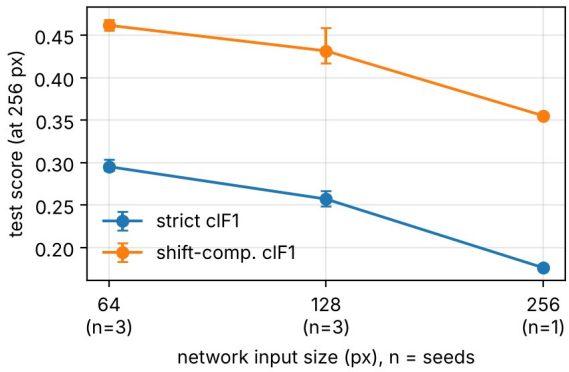


Figure 3. Strict and shape-compensated centreline F1 at three input sizes (mean over seeds, bars = min to max). All are scored at 256 px.

Accuracy falls as resolution rises: 0.295 at 64 px, 0.257 at 128 px, 0.176 at 256 px. Two mechanisms are consistent with this. With three pooling stages, the deepest features of a 64 px input each see an eighth of the

image, so the network reasons about the whole V rather than local bark texture. And a 10 px label offset at 256 px is only 2.5 px at 64 px, so the training signal is less contradictory. The 256 px result is a single seed.

6.3 Label re-registration, fork rule and sensitivity

- **Image-based re-registration (rejected).** Cross-correlating each mask with a bark map gave ambiguous peaks (best peak only ≈ 1.09 × the runner-up). On 12 inspected trees it corrected about 9 but pulled 3 onto grey trellis posts, so labels were not auto-corrected.
- **Model-based re-registration (v2r).** Each training and validation mask was shifted to best match the v1 prediction, accepted only if it raised centreline F1 by at least 0.05. 353 of 465 masks moved, by a median of 10 px in both directions, consistent with a tree-specific displacement rather than a fixed crop error. Test labels were never changed.
- **Fork rule.** The first rule (first row where two runs of mask appear) moved 13 px when a mask was thickened by 8 px; the skeleton-junction rule moves 5 px under the same test (now a unit test) and cut the v1 median fork error from 29 to 10 px.
- **Training-set size.** 100, 199 and 398 training trees gave strict F1 of 0.260, 0.246 and 0.257. More trees did not help, which points to label quality rather than quantity as the bottleneck.
- **Tolerance and baseline parameters.** At 2, 4 and 8 px the selected model scores 0.16, 0.29 and 0.51 and the baseline 0.06, 0.12 and 0.23, so the ranking does not depend on tolerance. The top ten of 75 baseline grid points score 0.116 to 0.124, so the baseline is not under-tuned.

6.4 Runtime, FLOPs and the roofline

Profiling one tree on one thread (cProfile plus timers; results/profile.json): reading two PNGs 9.3 ms (57%), crop and normalise 0.6 ms (4%), U-Net forward at 64 px 6.0 ms (36%), fork and angle 0.6 ms (3%). Profiling was run on my laptop (Windows 11, Intel CPU, 10 threads).

The network costs 0.29 GFLOP at 64 px, 1.17 at 128 px and 4.68 at 256 px (2 FLOP per multiply-accumulate, counted by estimate_flops and checked against a hand count). Its arithmetic intensity is 43 to 58 FLOP/byte. The laptop's machine balance is 111 GFLOP/s ÷ 23.3 GB/s ≈ 4.8 FLOP/byte, measured with a matrix multiply and an array copy, so the network is compute-bound (Figure 4). It reaches 50 to 66 GFLOP/s, 45% to 59% of the single-thread ceiling. A second thread gives only a modest gain at 64 px (5.8 to 4.4 ms) because each layer's work is small, but halves the time at 256 px (71 to 37 ms).

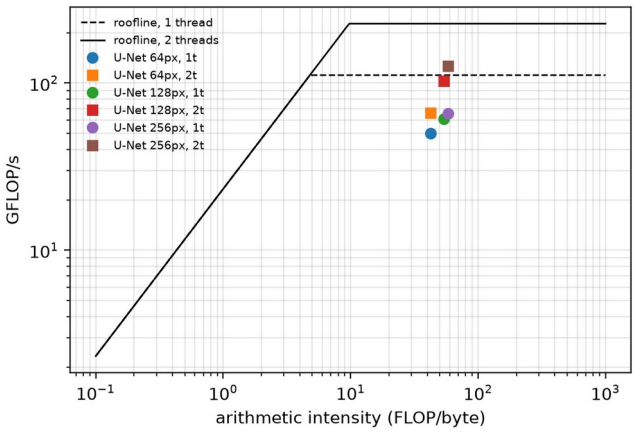


Figure 4. Roofline of my laptop CPU (matrix-multiply ceiling and copy bandwidth) with the U-Net at 64, 128 and 256 px on 1 and 2 threads.

The selected model needs 1.9 MB of weights and about 1.9 MB of activations and trains in 2.5 to 4.5 minutes on 2 CPU cores. The baseline takes 65 ms per tree, mostly in the three-scale Hessian, so it is 11× slower than the network for less than half the accuracy. File decoding is now the largest cost; on a robot the frames arrive from the camera in memory, so per-tree latency would be about 7 ms.

7. Limitations, failure modes and lessons learned

The largest limitation is the labels, not the method: until the masks are registered to the images they are scored against, no number in this report is an absolute measure of accuracy.

- **Label registration.** Masks are displaced by a tree-specific amount (median about 10 px at 256 px). Strict scores are capped and the shape score is optimistic. **[JUDE: run the trunk check (doc/trunk_annotator.html, then scripts/trunk_check.py, about 10 min) and replace this bracket with one sentence of the result, e.g. "On N test trees I clicked the visible trunk at one image row; the hand label sat a median X px from it and the selected model Y px." If you do not run it, delete the bracket.]**

- **Fork height misses its criterion** (median 11 to 19 px against 10 px). The predicted fork sits where the predicted leaders separate, which depends on mask width, and the masks are still about twice as wide as the branches.
- **A template prior.** On trees whose V is asymmetric or leaning, the network draws a more typical V (Figure 2, R15N28). Its output is partly a learned average tree, not only what the pixels show.
- **Relative 3-D only, one orchard, one camera.** Intrinsics for the back-projection are assumed and nothing shows transfer to other cultivars, trellis forms, lighting or sensors. The 256 px result is one seed and the depth benefit at 64 px rests on 3 seeds.

Unsupported inputs and failure modes. Trees that are not open-V still get a fork and angle for the two largest branches unless no junction qualifies (then None). A large missing-depth area (> 30% of pixels) makes the depth channel uninformative and the model falls back on colour; the tool does not yet flag this, although the audit computes the fraction and the check is one line. A trellis post near the trunk is marked by the baseline and occasionally merged into the trunk by the U-Net. Images that are not 640 × 480, or RGB and depth of different sizes, raise ValueError before processing (tested). A different camera or crop is outside the validated scope.

Most vulnerable numerical steps. The least-squares angle fit would become ill-conditioned for near-horizontal leaders (not present on this trellis). Sub-pixel peak interpolation assumes a locally parabolic peak; on flat or double peaks the peak ratio is near 1 and that alignment is not trusted. Early stopping on a noisy validation score made one v2 seed stop after 1 epoch; a longer patience or smoothed validation curve would make training more robust.

Lessons learned. (1) Audit the data before modelling: the two most important findings, depth direction and label displacement, came from the audit and changed how every result is read. (2) Look at predictions, not only scores: the over-wide v1 masks and the template-like V were obvious in one figure and invisible in the mean F1. (3) With noisy labels, higher resolution was worse; the smaller input gave a model that was more accurate, faster and more repeatable. (4) Measure what the robot needs: pixel IoU ranked methods similarly but said nothing about the angle or fork, which are what a planner uses.

Proposed improvements. Ask the teaching team whether the masks were drawn on another capture and, if so, register them once with a few manual control points per tree; predict the centreline directly (a distance-transform target) to remove the mask-width bias in fork height; and use the winter images, where branches are visible, to generate labels that are aligned by construction.

8. AI use and reflection

[JUDE: this section must be true and in your words. Everything not in a yellow bracket is taken from doc/AI_USE_LOG.md. Read each sentence, delete anything that is not accurate, and fill every bracket. Remove all yellow brackets before submitting.]

Tools. Claude (Anthropic), used through the Claude app with browser and computer access, between 7 and 14 October 2026, and again on 23 October to check the repository against the brief, condense the report to the page limit, and make the final clean-up commits and pull request (merging the history, removing empty logs, regenerating the test report and docs, and re-running the baseline and profiling on my laptop).

What it was used for and the approximate level of contribution.

Task	AI contribution	My contribution
Collecting the brief, dates and staff answers from Moodle and edStem	High	Chose the dataset; checked the summary against the brief PDF
Data audit code and first findings	High	**[what you checked by eye, e.g. opened N random trees]**
Library code, tests and experiment scripts	High (first drafts)	**[what you re-ran, read line by line or changed]**
Choice of methods, metrics, split and acceptance criteria	Proposed	**[which you accepted, rejected or changed, and why]**
Report	First draft and condensing to 10 pages	**[sections you rewrote]**

Overall I estimate the AI wrote **[X]**% of the code and **[Y]**% of the first-draft text. **[One sentence on why you used AI for these tasks, e.g. time available after choosing the project in week 10 and wanting to spend it on understanding and checking rather than typing.]**

Errors and unhelpful output, and how they were caught.

1. The first filename parser silently skipped the 42 renamed row-4 winter files. Caught because the audit printed an "unparsed" count; fixed and covered by a test.
2. It first assumed bright depth = near. The audit showed the opposite in 99.6% of trees.
3. Its first registration reference (ridge texture) gave ambiguous peaks, and the colour reference was pulled onto trellis posts, so automatic label correction was rejected.
4. Analysis scripts indexed a compressed .npz inside a loop, decompressing the whole array each time; the baseline run was killed at 5 GB of memory.
5. The first training configuration produced masks about three times too wide, caught by looking at predictions rather than summary numbers; its first fork rule was biased by mask width, found with a deliberate thickening test.
6. It identified the 64 px model while looking at test results; the choice was re-checked on validation rows only before adoption.
7. It wrote a journal name for the source paper it could not verify (replaced by the arXiv reference), and a draft claimed the tool flags missing depth, which the code does not do (corrected).

How AI output was checked. [e.g. re-ran every script from a clean clone and compared numbers with results/summary_table.csv; ran pytest; read every function I present in the demo; compared N trees by eye; opened each reference.]

Decisions that stayed mine. [e.g. the dataset choice; not auto-correcting the labels; the acceptance thresholds; accepting 64 px RGB-D as the final model; what to claim and what to call a limitation.]

Effect on my understanding and workflow. [3 to 4 honest sentences: what you now understand that you did not (e.g. why IoU fails for thin structures, row-grouped splits, roofline analysis), and what you would do differently next time.]

References

1. Chen, Z., Ting, D., Newbury, R., Chen, C. (2020). Semantic segmentation for partially occluded apple trees based on deep learning. arXiv:2010.06879. Source of the TreeID dataset and of the occluded-branch recall metric.
2. Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. MICCAI. arXiv:1505.04597.
3. Sato, Y. et al. (1998). Three-dimensional multi-scale line filter for segmentation and visualization of curvilinear structures in medical images. Medical Image Analysis 2(2), 143 to 168.
4. Woebbecke, D. M., Meyer, G. E., Von Bargen, K., Mortensen, D. A. (1995). Color indices for weed identification under various soil, residue, and lighting conditions. Transactions of the ASAE 38(1), 259 to 269.
5. Csurka, G., Larlus, D., Perronnin, F. (2013). What is a good evaluation measure for semantic segmentation? BMVC.
6. Williams, S., Waterman, A., Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM 52(4), 65 to 76.
7. MMA3001 unit notebooks: 2.1 Optimisation and profiling; 3.1 to 3.6 Interpolation; 5.1 to 5.6 Machine-learning regression, evaluation and data selection.
8. Software: NumPy, SciPy, scikit-image, OpenCV, PyTorch, pandas, Matplotlib, pytest, pdoc.

[JUDE: refs 3 to 6 were written from memory by the AI. Open each and confirm volume and pages before submitting.]

Appendix A. Qualitative results (not counted in the page limit)



Figure 2. Five test trees (rows). Red outline: hand label. Cyan: prediction. Columns: summer image with label, baseline, U-Net RGB-D 128 px (v1), U-Net trained on re-registered labels (v2r), selected U-Net RGB-D 64 px. Titles give strict / shape F1.

Appendix B. Reproducing the results

The README gives the environment and the command for every result: build_cache.py (cached arrays), run_audit.py, label_noise_study.py, run_baseline.py, queue.sh / refine_labels.py / queue_v2.sh / queue_v3.sh (all U-Net runs), validation_selection.py, analyse.py (tables, statistics and figures), profile_pipeline.py, trunk_check.py, then pytest and pdoc. Per-run logs, per-tree test metrics (results/runs/*/test_metrics_full.csv), the full audit (results/audit.json) and the baseline grid (results/runs/baseline/) are in the repository.